IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Presented at Review 2 of 3, BCSE307L Compiler Design, Vellore Institute of Technology, 2026.

Project repository: https://github.com/Shlok1107141/Compdes-Syntax-table · *Live site:* https://shlok1107141.github.io/Compdes-Syntax-table/

REVIEW 2

DenseSym: A Compressed Compiler Symbol Table Using Patricia Tries, 16-Bit Arena Indices, and Bit-Packed Metadata

Review 2: Web Application, Evaluation on Real C Code, and Selection of the Best Combination

SHLOK (24BCI0061), SAGNIK DATTA (24BCT0102), HARDIK (24BCI0017), AND EBEN (24BCE2459)
School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Vellore, Tamil Nadu 632014, India

ABSTRACT Review 1 showed, from terminal runs of eight symbol-table structures, that a compressed table combining a Patricia trie, 16-bit arena indices and bit-packed attributes needs 75.4% less memory than a textbook chained hash table on a 30-identifier workload. This second review turns that engine into a working system and evaluates it on real code. A C front end tokenizes source, tracks scopes and extracts each declaration's type, scope, storage class and offset; all eight structures are built and verified for every request. A FastAPI service runs the analyzer as a bounded child process that passed all 25 cases of an adversarial test suite, and a React web application shows the metrics, an ablation of all seven combinations, the symbol table with each 32-bit packed word, and the trie. On 84 preprocessed translation units from five open-source C projects (126,374 declarations) the full combination stores the tables in 2.48 MB instead of 9.63 MB: 74.2% less, or 20.5 instead of 76.6 B per declaration. Arena allocation with bit-packing alone saves 73.8%, while the trie alone makes the table 58.0% larger. The full combination is the smallest structure in 44 of the 84 files and arena with bit-packing in the other 40; the latter inserts four to five times faster. Results are stable across random seeds (spread of at most one percentage point). We select the full combination as the system's compressed table, keep arena with bit-packing as the faster alternative, and outline deployment on ESP32 and Arduino Uno boards for Review 3.

INDEX TERMS Ablation study, compiler front end, memory optimization, Patricia trie, symbol table, web application.

I. INTRODUCTION

Review 1 presented DenseSym, a compiler symbol table that stores a Patricia trie of 10-byte nodes in arenas addressed by 16-bit indices and packs each declaration's attributes into one 32-bit word [1], [2]. Measured from the terminal, it needed 669 B instead of 2,725 B for the prototype's 30 identifiers (75.4% less); the ablation showed that arena allocation with bit-packing alone reached 661 B and that the trie alone made the table 48.1% larger. Those measurements had three limits: the identifiers were a fixed list or generated rather than parsed from real programs, the results could only be read in a terminal, and speed was not measured.

This review removes those limits. Its contributions are:

1) a C front end that reads arbitrary C source, tracks lexical scopes, extracts every declaration, builds and verifies all eight structures, resolves each identifier use through the compressed table's scoped lookup, and reports the peak memory when block scopes are released;
2) a REST service (`POST /analyze`) that runs the analyzer as a sandboxed child process with bounded input size, request rate, concurrency, time and output, tested against 25 hostile inputs and request floods;
3) a web application in which any C program can be analyzed and its symbol table, trie and ablation inspected;
4) an evaluation on 93 real C files from five projects, a speed benchmark and a five-seed stability check, from which the best combination of techniques is selected.

Section II summarizes related work. Section III describes the system and its user interface, Section IV reports the results and the selection of the best combination, and Section V outlines the hardware work planned for Review 3.

II. RELATED WORK

Review 1 surveyed fifteen works in detail; we summarize them here. Compressed tries store shared prefixes once [3]–[7] but

This work is the team project of the course BCSE307L Compiler Design (Slot G2+TG2), School of Computer Science and Engineering, Vellore Institute of Technology, Vellore.

link heap-allocated nodes with pointers. Allocator studies [8], [9] and region-based memory management [10] show that per-object headers and fragmentation dominate the cost of many small objects. Production compilers use engineered hash containers tuned for throughput [11], and the standard analyses of hashing and search trees [12]–[14] give asymptotic rather than byte-level costs. Static allocation for small devices is established in sensor-network languages [15], and compiler-instrumented sanitizers [16] make memory-safety checks routine. Table 1 summarizes the properties that decide a symbol table's memory; DenseSym is the only structure with all four.

TABLE 1. Properties of Related Structures and of DenseSym

Structure	Shared prefixes	No per-object malloc	16-bit links	Packed attributes
Chained hash table [1], [13]	no	no	no	no
PATRICIA [3]	yes	no	no	no
Burst trie [5]	yes	partly	no	no
HAT-trie [6]	yes	partly	no	no
Adaptive radix tree [7]	yes	no	no	no
Regions / arenas [9], [10]	no	yes	no	no
Ternary search tree [14]	yes	no	no	no
DenseSym (this work)	**yes**	**yes**	**yes**	**yes**

"Partly": the burst trie and HAT-trie keep several strings in one container, so they allocate less often than once per string, but their nodes are still heap objects linked by pointers.

III. PROPOSED SYSTEM

A. ARCHITECTURE

The system has three decoupled tiers (Figure 1). The C engine parses source text and builds the structures; a Python service built with FastAPI [17] bounds every request and runs the engine as a child process; and a React [18] application written in TypeScript presents the result. The submitted code is only parsed, never compiled or executed.

1) ENGINE FRONT END

The analyzer (`symtab_analyze.c`) tokenizes the source and tracks scope depth: `{` opens a scope and `}` closes one. For each declaration it records one of 16 type codes (int, char, pointer, array, struct, function, . . .), one of eight storage classes (local, global, static, param, extern, register, typedef, member) and an offset. All eight structures of Review 1 are then built from the declarations, with the same variant code, and every declaration is looked up in every structure before any number is printed. Each identifier use is resolved through the compressed table's scoped lookup: the declarations of that name are tried newest first and the first whose scope is still open wins, which gives the reference count of each symbol. Finally the analyzer replays the scope exits to report the peak memory a compiler would need if each block's symbols were released when the block closed. The result is one JSON object on standard output.

TABLE 2. Analyzer Limits (All Input Is Untrusted)

Limit	Value
Source text	1 MiB
Tokens	200,000
Declarations	8,192
Identifier length	255 characters
Address space / CPU (self-imposed)	256 MiB / 5 s
Exit codes	0 ok, 2 input rejected, 3 verification failure

2) BACKEND SERVICE

The service exposes one endpoint, `POST /analyze`, whose request is `{"code": "..."}`. Table 3 lists the bounds it enforces. The child process runs with no shell, a minimal environment and an empty temporary directory, so it cannot reach the rest of the machine. For the four-declaration program `int sensor_temp; int sensor_press; void system_init() { int local_val; }` the response reports 815 B for the baseline and 121 B for the compressed table (85.15% less), together with the symbol table, the ablation of all seven combinations and the peak figures.

TABLE 3. Bounds Enforced by the Backend

Check	Limit	Result when exceeded
Request size	64 KB	HTTP 413
Request rate	30 per minute per client	HTTP 429
Concurrent analyses	4	requests wait
Analyzer run time	5 s	HTTP 504
Analyzer output	8 MB	HTTP 500
Input rejected (exit 2)	—	HTTP 400
Origins (CORS)	allow-list	blocked

Figure 2 traces one request from the browser through these checks, the trie inserts and the verification pass to the rendered result.

B. WEB INTERFACE DESIGN

The interface (Figure 3) has a C source editor on the left, with built-in presets, a 64 KB size meter and tab indentation, and three views on the right. A switch selects the mode: in live mode the editor's code is sent to the engine; offline mode shows the engine's own precomputed results for the built-in presets, so the site works without a backend.

- **Metrics & Ablation:** the memory saving, the baseline and compressed totals with bytes per symbol, a bar for each of the seven combinations (Figure 4), and the peak memory with scope release.
- **Symbol Table:** one row per declaration with its type, scope depth, storage class, offset, references and the 32-bit packed word in hexadecimal (Figure 5), with search and filters.
- **Radix Trie Visualizer:** the trie with each node's edge text, arena index and attached symbols, with zoom (Figure 6).

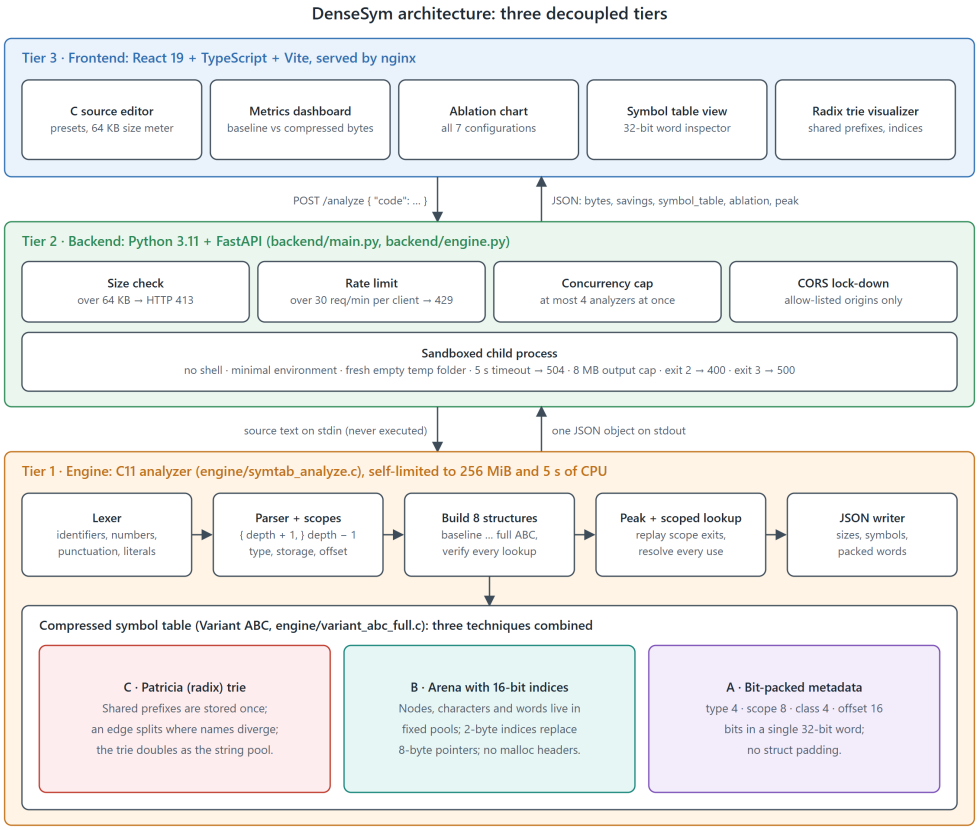


FIGURE 1. DenseSym architecture: React frontend, sandboxed FastAPI backend, and the C11 analyzer that builds all eight structures with the three techniques of the compressed table.

C. USER-INTERFACE DESIGN CHANGES

The interface went through several iterations between Reviews 1 and 2 (Table 4, from the repository's history). The main principle that emerged is that every number on screen must come from the engine: an early version estimated figures in the browser, and it was replaced by the engine's precomputed results.

TABLE 4. Interface Iterations Before Review 2

Date	Change	Reason
18 Sep	First web app: statistics dashboard and radix trie visualizer	make the engine's output visible
13 Oct	Live backend hardened: health detection, error states, offline fallback	work with or without the backend
13 Oct	Offline mode shows only engine-measured results for presets	no figure the engine did not produce
13 Oct	A rejected input no longer marks the engine offline	separate bad input (400) from an outage
13 Oct	Redesign: light theme, teal and orange accents, widescreen layout, metric tiles, progress timer	readability in demonstrations
13 Oct	Trie visualizer connector lines repaired	a correct drawing
14 Oct	Editor tab and multi-line indentation	editing real code in the browser

For custom code in offline mode the browser uses an approximate parser, and the interface labels it as such; live mode always uses the engine. The site also hosts a separate parse-table compression suite developed in parallel, which is outside the scope of this paper.

D. SECURITY AND ADVERSARIAL TESTING

Because the service accepts arbitrary source text from the browser, the backend and the analyzer were tested as a black box against hostile input (Table 5). The acceptance rule is strict: no input may produce a server error (HTTP 5xx), hang the service or trigger a sanitizer report, and every request must end in HTTP 200 or in a 4xx response that names the problem. The suite was run on 13 October 2026 against the analyzer built with AddressSanitizer and UBSan [16]; every one of the 25 cases passed, with no leak, overflow or undefined-behavior report.

E. EVALUATION METHODOLOGY AND METRICS

Table 6 lists the three datasets. The real code comes from SQLite extensions [19], GNU Readline [20], XZ Utils [21], GNU MPFR examples [22] and the engine itself. Each file is measured raw (its own declarations) and after preprocessing with `gcc -E` (everything a compiler's table holds, system headers included); 9 files could not be preprocessed because of missing headers.

Six metrics are reported: total bytes and the saving against the baseline; median bytes per declaration; the number of files in which each structure is the smallest; peak memory with scope release; nanoseconds per insert, lookup hit and lookup

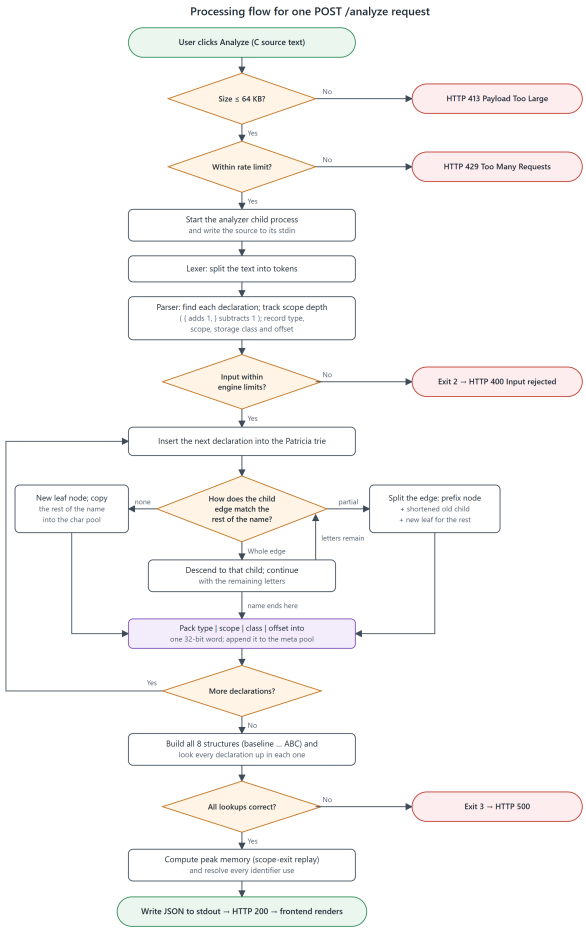


FIGURE 2. Processing flow for one POST /analyze request, including the three ways a trie insertion can match an existing edge and every rejection path.

TABLE 5. Adversarial Test Suites (Every Case Passed)

Suite	Cases	Inputs	Required outcome
Malformed requests	6	empty code, missing field, non-JSON body, number instead of text, GET on /analyze	200 for empty code; 422 or 405 otherwise
Size limits	3	77 KB source, 2 MiB body, 300-character identifier	413; 400 for the identifier
Hostile syntax	12	random bytes, 5,000 unclosed or stray braces, 5,000 nested parentheses, unterminated comment and string, control bytes, shell metacharacters, format strings, system("rm -rf /"), 3,000 redeclarations, 200 names sharing a 200-character prefix	200 within the time limit; nothing executed
Floods and origins	4	foreign Origin header, 50 concurrent requests, 60 rapid requests, health check after	no CORS grant; only 200 or 429; still healthy

TABLE 6. Datasets

Dataset	Content	Size
Synthetic sweep	3 naming patterns, n = 10 . . . 1,000, 7 combinations, seeds 1–5	840 runs
Real C, raw	93 files from 5 projects	7,437 declarations
Real C, preprocessed	84 translation units	126,374 declarations
Speed	insert, lookup hit, lookup miss; 8 structures; n = 100, 1,000	48 rows

miss; and the spread of the AB–ABC difference across five seeds. Every structure is verified before it is measured.

IV. RESULTS

A. THE SYSTEM AT WORK

Figure 3 shows the 30-identifier preset: 2,725 B for the baseline and 669 B compressed, a 75.45% saving and 4.1 times the density. The ablation bars of Figure 4 reproduce the terminal values of Review 1 exactly (for example AB 661 B and the trie alone 4,035 B), because the same variant code runs behind the web page. With scope release the largest table needed at any moment is 601 B instead of 2,432 B, since at most 26 of the 30 symbols are alive at once.

B. REAL C CODE

Table 7 gives the totals over the 84 preprocessed translation units and Figure 7 the savings in both modes. Arena allocation alone saves 58.1%; adding bit-packing raises this to 73.8%, and adding the trie on top to 74.2%. The trie alone makes the tables 58.0% larger.

TABLE 7. Memory on 84 Preprocessed C Translation Units (126,374 Declarations)

Structure	Total bytes	Saving
Baseline	9,631,762	—
A: bit-packing	8,620,770	10.5%
B: arena	4,039,050	58.1%
C: trie	15,214,461	−58.0%
AB	2,522,562	73.8%
AC	10,107,461	−4.9%
BC	4,000,603	58.5%
ABC (DenseSym)	**2,484,115**	**74.2%**

Per file, ABC is the smallest structure in 44 of the 84 preprocessed units and AB in the other 40; the median difference between them is 0.1% in ABC's favor (range −4.3% to +10.0%). The median memory per declaration falls from 76.6 B to 20.5 B. If block scopes are released on exit the median peak saving is 71.4%. On the raw files the baseline needs 573,678 B and AB and ABC about 121,500 B each (78.8%); ABC is the smallest in 49 of 93 files and AB in 44, and the median per declaration falls from 82.1 B to 17.7 B. Table 8 breaks the raw results down by project.

After preprocessing, the per-project saving of ABC ranges from 73.2% (SQLite extensions) to 76.9% (the engine); ABC beats AB by 7.5% on MPFR and 5.9% on XZ and trails it by 0.7% on Readline.

C. STABILITY ACROSS SEEDS

The synthetic sweep of Review 1 was repeated with seeds 1 to 5 (840 runs). Table 9 gives the difference ABC − AB in percentage points of the baseline (positive means ABC is smaller). The spread between seeds is at most one point, far smaller than the differences being reported.

D. SPEED

Figure 8 and Table 10 give the time per operation at n = 1,000 (Linux, glibc, `-O2`). Removing `malloc` from the insert path

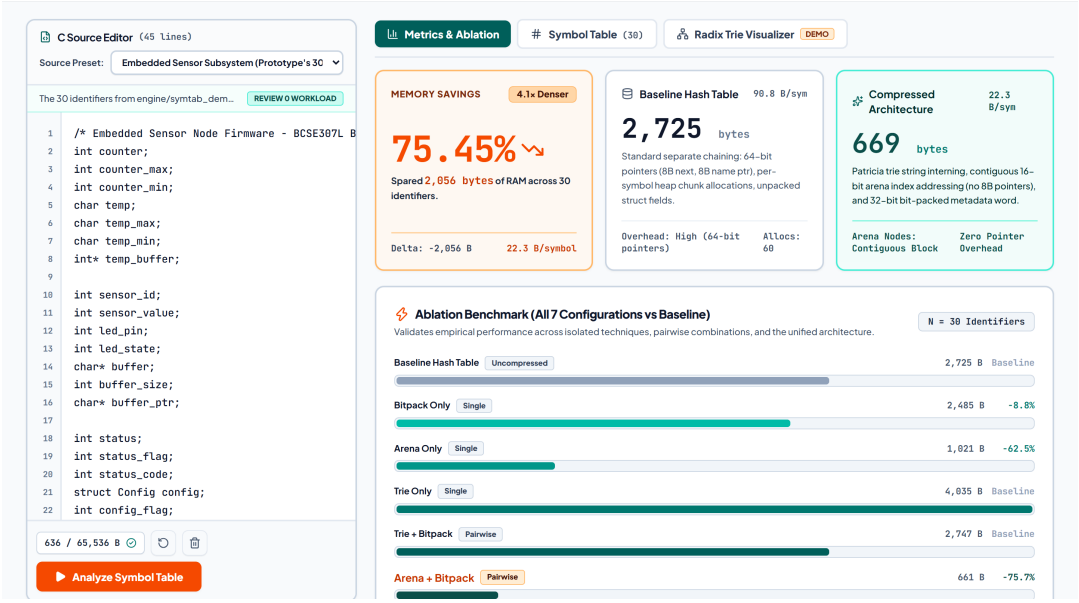


FIGURE 3. The web workbench analyzing the 30-identifier preset: editor, memory saving (75.45%), baseline 2,725 B and compressed 669 B, and the start of the ablation.

TABLE 8. Raw Source Files by Project

Project	Files	Decls	Baseline B	AB B	ABC B	ABC saving	ABC vs AB
GNU MPFR [22]	6	55	6,799	1,415	1,015	85.1%	+28.3%
GNU Readline [20]	17	345	33,262	7,414	6,709	79.8%	+9.5%
SQLite ext. [19]	55	5,945	448,788	94,748	94,938	78.8%	−0.2%
DenseSym engine	10	990	75,058	15,778	17,167	77.1%	−8.8%
XZ Utils [21]	5	102	9,771	2,139	2,007	79.5%	+6.2%

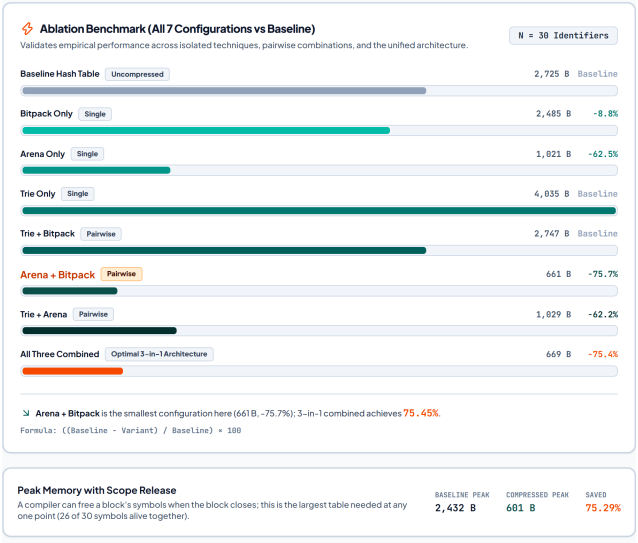


FIGURE 4. Ablation bars for the seven combinations and the peak memory with scope release (2,432 B against 601 B).

TABLE 9. ABC − AB in Percentage Points, Mean ± SD Over Five Seeds

Pattern	n = 10	n = 50	n = 100	n = 300	n = 1,000
Realistic	+6.7 ± 0.7	−0.2 ± 0.7	−2.5 ± 0.5	−3.3 ± 0.4	−2.6 ± 0.2
Random	+3.5 ± 0.6	−5.9 ± 0.2	−6.9 ± 0.2	−7.1 ± 0.2	−8.4 ± 0.1
Scoped	+5.6 ± 0.4	−1.1 ± 0.9	−0.8 ± 0.2	+0.1 ± 0.3	+0.8 ± 0.1

makes AB three to four times faster to insert into than the baseline. The trie gives part of that back through edge splits and sibling scans: ABC is 1.3 to 1.5 times slower than the baseline

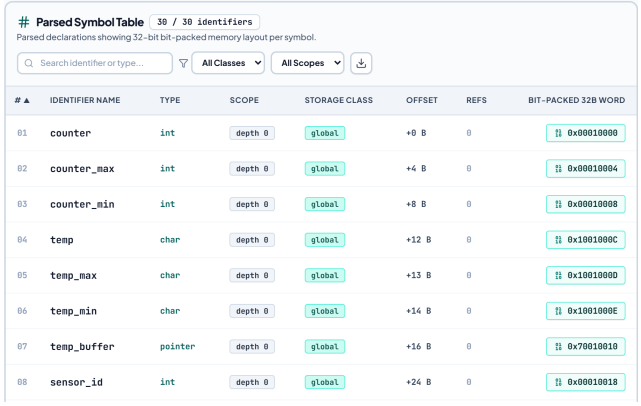


FIGURE 5. Symbol Table view (first 8 of 30 rows): each declaration with its packed 32-bit word.

on successful lookups, but faster on misses for realistic and scoped names because a trie rejects a non-matching prefix early. In absolute terms the difference is negligible: a typical translation unit, about 1,500 declarations and a few thousand lookups, spends well under a millisecond in its symbol table either way.

E. WHICH COMBINATION IS BEST

Table 11 compares the candidates on every metric. Bit-packing alone (A) saves about 10% and the trie alone (C) always costs memory, so neither is a candidate; the arena alone (B) saves 15 to 16 points less than ABC.

The best combination for memory is all three techniques

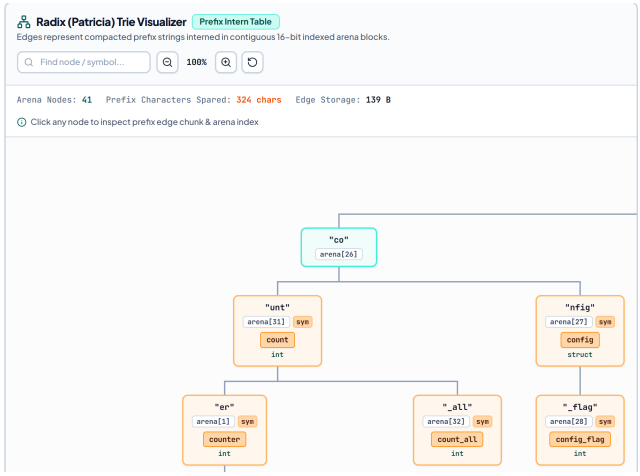


FIGURE 6. Radix Trie Visualizer: 41 arena nodes and a 139-byte edge pool for the preset.

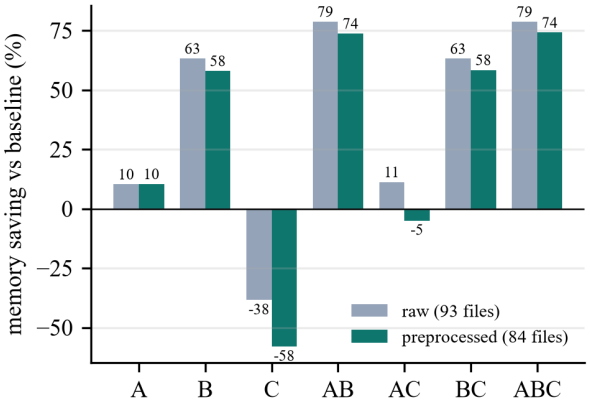


FIGURE 7. Memory saving on real C code, raw (93 files) and preprocessed (84 translation units).

TABLE 10. Nanoseconds per Operation at n = 1,000

Workload	Structure	Insert	Hit	Miss
Realistic	Baseline	27.0	36.3	66.7
	AB	**7.8**	38.6	56.0
	ABC	39.6	46.6	**49.4**
Random	Baseline	29.4	38.0	69.4
	AB	**9.0**	38.2	55.6
	ABC	43.6	57.5	58.2
Scoped	Baseline	27.7	22.8	62.9
	AB	**6.8**	20.2	53.1
	ABC	27.4	32.4	**34.9**

(ABC). It has the smallest total on the preprocessed corpus (74.2% saving), is the smallest structure in more files than any alternative (44 of 84 preprocessed, 49 of 93 raw), has the highest mean saving on scoped programs, where names are redeclared, and rejects absent names fastest on realistic and scoped names. It is therefore the structure the system reports as "compressed".

Arena with bit-packing (AB) is a close second: within 0.4 points on real code, ahead on random names and four to five times faster to insert into than ABC. Where insert speed or simplicity matters more than the last one to two percent, AB is the better engineering choice. The deeper result is that arena allocation with bit-packing carries almost all of the saving (73.8% of the 74.2%), and that the trie is worth adding only inside the arena, where its nodes are 10 bytes.

F. DISCUSSION AND LIMITATIONS

- **Host model.** Every figure assumes a 64-bit host with 8-byte pointers and 16-byte allocator headers. On a 32-bit or 16-bit target both shrink, so the arena's advantage should shrink too; this has not yet been measured.
- **Corpus.** The real code is desktop libraries, not firmware; an embedded corpus would test the motivating use case more directly.
- **Baseline.** The baseline uses a fixed 64-bucket table as in the prototype; a resizing table would have shorter chains but a larger bucket array.
- **Limits.** The 16-bit indices cap a table at 65,535 nodes and 64 KiB of characters. The largest unit measured (SQLite `spellfix.c`, 2,755 declarations) used about 39.6 KB for nodes and characters, a 1.6-fold headroom; a single 8 MB amalgamation would need 32-bit indices, which give back part of the saving.

V. FUTURE WORK: HARDWARE DEPLOYMENT

The motivating setting for a compact symbol table is a small device, so Review 3 will move the engine onto two microcontroller boards: an ESP32 (32-bit, about 520 KB of SRAM) and an Arduino Uno (8-bit ATmega328P, 2 KB of SRAM). The plan is to:

1) make the engine portable to these targets (configurable pool sizes, no dependence on the host's C library) while keeping the PC results byte-identical;
2) run the eight structures on the boards with the same symbol sets and verification, measuring RAM with the boards' own heap counters and time with their hardware timers;
3) measure how the saving changes with 4-byte (ESP32) and 2-byte (Uno) pointers, and how many identifiers each structure holds before the board runs out of memory;
4) store the compressed table in the boards' non-volatile memory and reload it after a reset, which a pointer-free arena allows without rebuilding;
5) connect the boards to the web application so that a program typed in the browser is analyzed on the hardware.

VI. CONCLUSION

DenseSym is now a working system: a C front end that builds and verifies eight symbol-table structures for any C program, a sandboxed REST service and a web application that shows the result. On 126,374 real declarations the full combination of Patricia trie, 16-bit arena indices and bit-packed attributes needs 74.2% less memory than a textbook chained hash table (20.5 instead of 76.6 B per declaration), and it is the smallest structure in most files. Arena allocation with bit-packing alone achieves 73.8% and inserts four to five times faster, so it remains a strong alternative; the trie pays off only inside the arena.

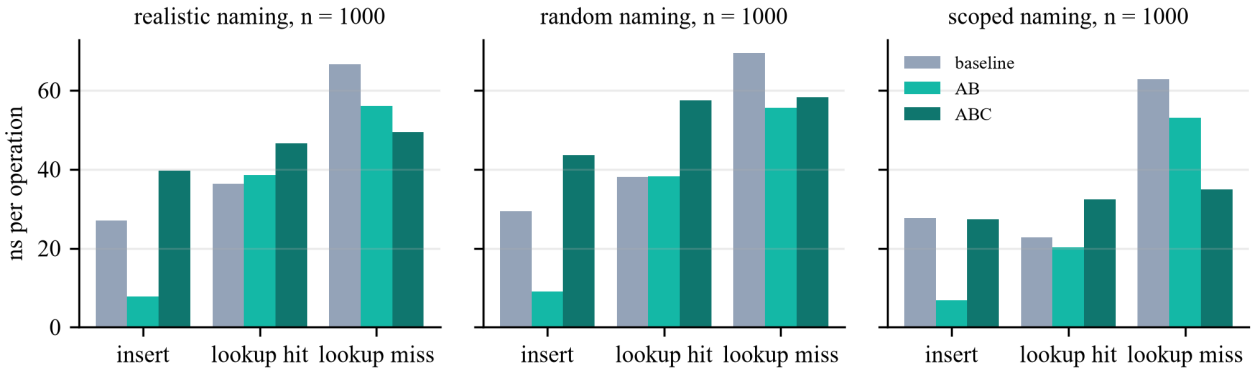


FIGURE 8. Time per operation at n = 1,000 for the baseline, AB and ABC (bench_results.csv).

TABLE 11. Selecting the Best Combination

Metric	Baseline	B	AB	ABC	Best
Real code, preprocessed: bytes (saving)	9,631,762	4,039,050 (58.1%)	2,522,562 (73.8%)	**2,484,115 (74.2%)**	ABC
Real code, raw: bytes (saving)	573,678	210,738 (63.3%)	**121,494 (78.8%)**	121,836 (78.8%)	AB, by 342 B
Smallest structure, preprocessed files (of 84)	0	0	40	**44**	ABC
Smallest structure, raw files (of 93)	0	0	44	**49**	ABC
Synthetic mean saving, scoped names	—	63.3%	78.2%	**78.6%**	ABC
Synthetic mean saving, random names	—	58.5%	**72.2%**	66.9%	AB
Insert, ns (realistic, n = 1,000)	27.0	8.7	**7.8**	39.6	AB
Lookup hit, ns	**36.3**	38.3	38.6	46.6	Baseline
Lookup miss, ns	66.7	69.3	56.0	**49.4**	ABC

CODE AND DATA AVAILABILITY

The analyzer, the backend and its test suites, the web application and the evaluation scripts are public at https://github.com/Shlok1107141/Compdes-Syntax-table, and the web application runs at https://shlok1107141.github.io/Compdes-Syntax-table/. In `engine/`, `make bench` produces Table 10 and Figure 8, and `real_corpus.py` produces Tables 7 and 8 and Figure 7; the adversarial suite of Table 5 is `backend/tests/test_hostile_inputs.py`.

ACKNOWLEDGMENT

The authors thank the faculty of the School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, for their guidance in the BCSE307L Compiler Design course.

REFERENCES

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006.

[2] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 2nd ed. San Francisco, CA, USA: Morgan Kaufmann, 2011.

[3] D. R. Morrison, "PATRICIA—Practical algorithm to retrieve information coded in alphanumeric," *J. ACM*, vol. 15, no. 4, pp. 514–534, Oct. 1968.

[4] E. Fredkin, "Trie memory," *Commun. ACM*, vol. 3, no. 9, pp. 490–499, Sep. 1960.

[5] S. Heinz, J. Zobel, and H. E. Williams, "Burst tries: A fast, efficient data structure for string keys," *ACM Trans. Inf. Syst.*, vol. 20, no. 2, pp. 192–223, Apr. 2002.

[6] N. Askitis and R. Sinha, "HAT-trie: A cache-conscious trie-based data structure for strings," in *Proc. 30th Australas. Comput. Sci. Conf. (ACSC)*, 2007, pp. 97–105.

[7] V. Leis, A. Kemper, and T. Neumann, "The adaptive radix tree: ARTful indexing for main-memory databases," in *Proc. IEEE 29th Int. Conf. Data Eng. (ICDE)*, 2013, pp. 38–49.

[8] P. R. Wilson, M. S. Johnstone, M. Neely, and D. Boles, "Dynamic storage allocation: A survey and critical review," in *Proc. Int. Workshop Memory Manage. (IWMM)*, 1995.

[9] E. D. Berger, B. G. Zorn, and K. S. McKinley, "Reconsidering custom memory allocation," in *Proc. ACM Conf. Object-Oriented Program., Syst., Lang., Appl. (OOPSLA)*, 2002, pp. 1–12.

[10] M. Tofte and J.-P. Talpin, "Region-based memory management," *Inf. Comput.*, vol. 132, no. 2, pp. 109–176, 1997.

[11] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *Proc. Int. Symp. Code Gener. Optim. (CGO)*, 2004, pp. 75–86.

[12] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1998.

[13] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[14] J. L. Bentley and R. Sedgewick, "Fast algorithms for sorting and searching strings," in *Proc. 8th ACM-SIAM Symp. Discrete Algorithms (SODA)*, 1997, pp. 360–369.

[15] D. Gay, P. Levis, R. von Behren, M. Welsh, E. Brewer, and D. Culler, "The nesC language: A holistic approach to networked embedded systems," in *Proc. ACM SIGPLAN Conf. Program. Lang. Design Implement. (PLDI)*, 2003, pp. 1–11.

[16] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "AddressSanitizer: A fast address sanity checker," in *Proc. USENIX Annu. Tech. Conf.*, 2012, pp. 309–318.

[17] S. Ramírez, "FastAPI." [Online]. Available: https://fastapi.tiangolo.com

[18] Meta Platforms, Inc., "React." [Online]. Available: https://react.dev

[19] "SQLite." [Online]. Available: https://www.sqlite.org

[20] C. Ramey, "The GNU Readline library." [Online]. Available: https://tiswww.case.edu/php/chet/readline/rltop.html

[21] L. Collin, "XZ Utils." [Online]. Available: https://tukaani.org/xz/

[22] "The GNU MPFR library." [Online]. Available: https://www.mpfr.org



SHLOK is an undergraduate student with the School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Vellore, India. In the DenseSym project, Shlok is responsible for the engine and the backend: the arena, bit-packed and trie data structures behind one C interface, the prototype and its verification harness, the C front end that extracts declarations with their scopes, the sandboxed FastAPI service that runs it, and the measurements on real C code.



SAGNIK DATTA is an undergraduate student with the School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Vellore, India. In the DenseSym project, Sagnik is responsible for the frontend: the React, TypeScript and Tailwind CSS web application with its C source editor and presets, the live metrics and ablation dashboard, the symbol-table view that shows each packed 32-bit word, and the radix trie visualizer, as well as the parse-table compression suite hosted on the same site.



HARDIK is an undergraduate student with the School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Vellore, India. In the DenseSym project, Hardik is responsible for the ablation study: the seven engine variants that isolate each technique and each pair of techniques, the scaling sweep from 10 to 1,000 identifiers over three naming patterns, and the comparison tables that separate what each technique saves.



EBEN is an undergraduate student with the School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Vellore, India. In the DenseSym project, Eben is responsible for the research paper and its literature survey, the security policy for untrusted source code and the bounds on the analyzer process, and the adversarial test suite that checks the service against malformed, oversized and hostile input and against floods of requests.